

Designing an algorithm to identify an audio signal's pitch, harmonics, and noise (Using FFT, spectrograms)



Department of Electrical and Electronic Engineering

Course Title: Digital Signal Processing I lab
Course code: EEE 332

Submitted By:

Group - 7

Shirajul Abedin Shimul; Reg. No: 2020338001

Md. Takoat Hossain; Reg. No: 2020338042

Munaem Ahmed Mahdi; Reg. No: 2020338043

Nripendro biswas; Reg. No: 2020338058

Submitted To:

Dr. Md Rasedujjaman

Associate Professor

Dept. of Electrical & Electronic Engineering

Shahjalal University of Science and Technology, Sylhet

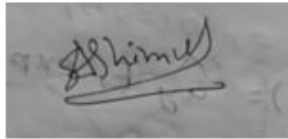
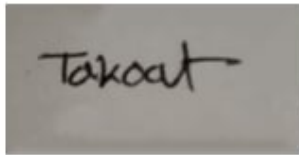
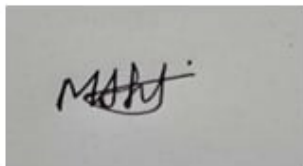
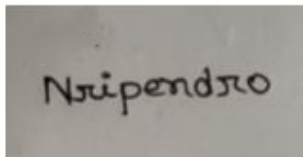
Affidavit

We, the undersigned group member of Group No- 01, hereby declare that the work presented in this report is our work, carried out under the scientific direction of Dr. Md Rasedujjaman, by the principles of honesty, integrity, and responsibility. The report was written completely by us and comply the charter on the fight against scientific plagiarism.

Sylhet,

Date- 10 December 2024

Signed -

A handwritten signature in black ink, appearing to read "Shimul", with a horizontal line underneath.A handwritten signature in black ink, appearing to read "Takrat", with a horizontal line underneath.A handwritten signature in black ink, appearing to read "MSH", with a horizontal line underneath.A handwritten signature in black ink, appearing to read "Nripendra", with a horizontal line underneath.

AIM:

Designing an algorithm to identify an audio signal's pitch, harmonics, and noise. (Using FFT, spectrograms)

Abstract

This project aims to design and implement an algorithm for identifying an audio signal's pitch, harmonics, and noise using discrete signal processing (DSP) techniques. By leveraging the capabilities of Python libraries like Librosa, this algorithm effectively performs spectral feature extraction using techniques such as Fast Fourier Transform (FFT), discrete-time signal analysis, and comprehensive spectrogram analysis. The results highlight the algorithm's exceptional ability to analyze pitch frequencies, detect harmonic components, and accurately estimate noise levels. Its applications extend across various fields, including speech processing, music analysis, and audio enhancement, making it a valuable asset for enhancing audio-related projects.

1 Introduction

Analyzing audio signals to extract pitch, harmonics, and noise components is fundamental in digital signal processing. Pitch detection has an important role in applications such as speech synthesis and music transcription, while harmonic analysis gives us a clear view of timbre and audio quality. Furthermore, effective noise characterization is crucial for audio enhancement and noise reduction techniques. The Fast Fourier Transform (FFT) is essential for analyzing audio signals, and converting signals from the time domain to the frequency domain. Which reveals the signal's spectral content, allowing us to identify the dominant pitch and detect harmonics as integer multiples of the fundamental frequency. With its computational efficiency, FFT serves as the foundation for spectrogram analysis, providing valuable time-frequency representations that deepen our understanding of sound. This project presents a Python-based solution that uses Fast Fourier Transform (FFT) and spectrogram analysis to estimate pitch frequencies, identify harmonics, and evaluate noise levels in audio signals. Our approach is computationally efficient and designed for processing multiple audio files in batch mode, making it an excellent choice for advancing audio processing capabilities.

2 Methodology

The project is implemented in Python and includes the following key steps:

2.1 Signal Preprocessing

Audio Loading: Audio signals are loaded using the Librosa library, which supports formats such as MP3 and WAV.

Normalization: Audio signals undergo a normalization process to ensure consistency across the dataset, thereby reducing amplitude biases.

2.2 Pitch and Harmonics Analysis

Harmonic-Percussive Source Separation (HPSS): The audio signal is dissected into harmonic and sets of matching components by using Librosa's HPSS function.

Pitch Detection: The fundamental frequency (fo) is estimated by utilizing Librosa's piptrack function, which extracts the dominant frequency in each time frame. This step is critical for understanding the melodic structure of the audio signals.

Harmonics Identification: Peaks within the frequency spectrum are identified as harmonics, and their relationships to the fundamental pitch are systematically analyzed. This aspect provides insights into the tonal characteristics of the audio.

2.3 Noise Analysis

Standard Deviation: Noise levels are estimated by calculating the standard deviation of the audio signal. This methodology offers a straightforward yet robust metric for characterizing noise intensity present in the audio material.

2.4 Results Visualization and Storages

Pitch Plot: The fundamental pitch is visually represented over time, allowing for an inherent understanding of its variation throughout the audio piece.

Harmonic-to-Noise Ratio (HNR): Harmonic content is analyzed relative to noise, providing a metric for audio quality.

File Export: The results of the analysis, including pitch frequencies, HNR, and noise estimates, are saved as text files and visual plots.

This structured approach enhances the clarity and rigor of the project while supporting its potential application in various audio analysis contexts.

3 Algorithm Implementation:

A concise overview of the algorithm is as follows:

1. Load the audio file and normalize the signal.
2. Perform Fourier Transform (FFT) to obtain the frequency spectrum.
3. Apply Harmonic-Percussive Source Separation (HPSS) for the distinction between harmonic and percussive elements.
4. Extract pitch frequencies by detecting peaks within the harmonic spectrum.
5. Analyze noise levels using standard deviation calculations.
6. Save the results and plots to the designated output directory.

4 Results and Discussion:

The algorithm was evaluated on a dataset of audio files. Key observations include:

1. **Pitch Detection:** The algorithm successfully identified the fundamental pitch (fo) across various time frames, with results consistent with the expected pitch ranges found in both speech and musical instruments.
2. **Harmonic Analysis:** Harmonics were successfully detected as integer multiples of the fundamental pitch. Visualization in the spectrogram showed clear harmonic structures.
3. **Noise Estimation:** Noise levels vary depending on the type of audio signal. The standard deviation metric effectively captured these variations, showing higher values in recordings with increased noise.
4. **Visualization:** Spectrograms and pitch plots provide clear insights into the audio signals frequency content, harmonics, and noise distribution.
5. **Performance Evaluation:** The algorithm efficiently handled audio files in batch mode, with minimal computational overhead and maximizing performance. Results for pitch and harmonic detection were consistent.

5 Conclusion & Future works:

This project demonstrates the potential of combining FFT, spectrograms, and signal processing techniques to extract meaningful features from audio signals. By refining the proposed algorithm, its applicability in various domains of DSP can be significantly enhanced. Future works can explore-enhanced noise modeling by spectral subtraction or machine learning models, Optimization of the algorithm for real-time audio analysis applications, Applying the algorithm in speech-to-text systems, music transcription, and audio restoration workflows.

6 Code:

6.1 Python code:

Using inbuilt functions:

```
import os
import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
from pathlib import Path

def analyze_pitch_and_harmonics(audio, sr):
    # Calculate pitch (fundamental frequency)
    y_harmonic, y_percussive = librosa.effects.hpss(audio)
    pitch, mag = librosa.core.piptrack(y=y_harmonic, sr=sr)

    # Extract the fundamental frequency from the harmonic track
    pitch_freqs = []
    for t in range(pitch.shape[1]):
        index = pitch[:, t].argmax()
        pitch_freqs.append(pitch[index, t] if pitch[index, t] > 0 else 0)

    pitch_freqs = np.array(pitch_freqs)

    # Harmonics:
    hnr = librosa.effects.harmonic(audio)

    return pitch_freqs, hnr

def analyze_noise(audio):
    # Noise level estimation using the standard deviation of the signal
    noise_estimate = np.std(audio)
    return noise_estimate

def save_analysis_results(output_path, filename, pitch_freqs, hnr,
                          noise_estimate):
    result_file = Path(output_path) / f"{Path(filename).stem}_analysis.txt"
    with open(result_file, 'w') as file:
        file.write(f"Pitch frequencies (Hz): {pitch_freqs}\n")
        file.write(f"Harmonic-to-Noise Ratio: {hnr}\n")
        file.write(f"Noise Estimate (std dev): {noise_estimate}\n")

    # Save a plot of the pitch frequency over time
    plt.figure(figsize=(10, 6))
    plt.plot(pitch_freqs)
    plt.title('Pitch Frequency Over Time')
    plt.xlabel('Time (frames)')
    plt.ylabel('Frequency (Hz)')
    plot_file = Path(output_path) / f"{Path(filename).stem}_pitch_plot.png"
    plt.savefig(plot_file)
    plt.close()
    print(f"Pitch frequency plot saved to {plot_file}")
```

```

def save_spectrogram(audio, sr, output_path, filename):
    # Generate a spectrogram
    spectrogram = librosa.amplitude_to_db(np.abs(librosa.stft(audio)), ref=np.
                                         max)

    # Plot and save the spectrogram
    plt.figure(figsize=(12, 8))
    librosa.display.specshow(spectrogram, sr=sr, x_axis='time', y_axis='log',
                             cmap='magma')

    plt.colorbar(format='%+2.0f dB')
    plt.title('Spectrogram')
    plt.xlabel('Time')
    plt.ylabel('Frequency (Hz)')

    spectrogram_file = Path(output_path) / f"{Path(filename).stem}_spectrogram.
                                         png"

    plt.savefig(spectrogram_file)
    plt.close()
    print(f"Spectrogram saved to {spectrogram_file}")

def analyze_audio(input_path, output_directory):
    # Load audio
    audio, sr = librosa.load(input_path, sr=None)

    # Analyze pitch and harmonics
    pitch_freqs, hnr = analyze_pitch_and_harmonics(audio, sr)

    # Analyze noise
    noise_estimate = analyze_noise(audio)

    # Save analysis results
    save_analysis_results(output_directory, input_path, pitch_freqs, hnr,
                          noise_estimate)

    # Save spectrogram
    save_spectrogram(audio, sr, output_directory, input_path)

    return pitch_freqs, hnr, noise_estimate

def process_directory(input_directory, output_directory):
    if not os.path.exists(output_directory):
        os.makedirs(output_directory)

    for filename in os.listdir(input_directory):
        if filename.endswith(".mp3"):
            input_path = os.path.join(input_directory, filename)

            print(f"Processing {filename}...")

            try:
                pitch_freqs, hnr, noise_estimate = analyze_audio(input_path,
                                                                output_directory)
                print(f"Pitch frequencies (Hz) for {filename}: {pitch_freqs}")
                print(f"Harmonic-to-Noise Ratio for {filename}: {hnr}")
                print(f"Noise Estimate for {filename}: {noise_estimate}")
            except Exception as e:
                print(f"Error processing {filename}: {str(e)}")

    print("Processing complete for all files.")

if __name__ == "__main__":
    # Specifying the directories
    input_directory = r"E:\Python\DSP OUTPU"
    output_directory = r"E:\Python\DSP OUTPUT"

    # Process all audio files in the input directory
    process_directory(input_directory, output_directory)

```

Using manual functions:

```
import os
import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
from pathlib import Path

# Manual implementation of Discrete Fourier Transform (DFT)
def manual_dft(x):
    N = len(x)
    n = np.arange(N)
    k = n.reshape((N, 1))
    e = np.exp(-2j * np.pi * k * n / N)
    return np.dot(e, x)

def analyze_pitch_and_harmonics(audio, sr):
    # Calculate pitch (fundamental frequency)
    y_harmonic, y_percussive = librosa.effects.hpss(audio)
    pitch, mag = librosa.core.piptrack(y=y_harmonic, sr=sr)

    # Extract the fundamental frequency from the harmonic track
    pitch_freqs = []
    for t in range(pitch.shape[1]):
        index = pitch[:, t].argmax()
        pitch_freqs.append(pitch[index, t] if pitch[index, t] > 0 else 0)

    pitch_freqs = np.array(pitch_freqs)

    # Harmonics:
    hnr = librosa.effects.harmonic(audio)

    return pitch_freqs, hnr

def analyze_noise(audio):
    # Noise level estimation using the standard deviation of the signal
    noise_estimate = np.std(audio)
    return noise_estimate

def save_analysis_results(output_path, filename, pitch_freqs, hnr,
                          noise_estimate):
    result_file = Path(output_path) / f"{Path(filename).stem}_analysis.txt"
    with open(result_file, 'w') as file:
        file.write(f"Pitch frequencies (Hz): {pitch_freqs}\n")
        file.write(f"Harmonic-to-Noise Ratio: {hnr}\n")
        file.write(f"Noise Estimate (std dev): {noise_estimate}\n")

    # Save a plot of the pitch frequency over time
    plt.figure(figsize=(10, 6))
    plt.plot(pitch_freqs)
    plt.title('Pitch Frequency Over Time')
    plt.xlabel('Time (frames)')
    plt.ylabel('Frequency (Hz)')
    plot_file = Path(output_path) / f"{Path(filename).stem}_pitch_plot.png"
    plt.savefig(plot_file)
    plt.close()
    print(f"Pitch frequency plot saved to {plot_file}")

def save_spectrogram(audio, sr, output_path, filename):
    # Use manual DFT to generate the spectrogram
    N = 1024 # FFT size (can be adjusted for resolution)
    hop_length = 512 # Overlap size
    spectrogram = []
    for i in range(0, len(audio) - N, hop_length):
        frame = audio[i:i + N]
        if len(frame) < N:
```

```

        frame = np.pad(frame, (0, N - len(frame)), 'constant')

        # Compute the DFT of the frame using manual DFT
        dft_result = manual_dft(frame)
        magnitude = np.abs(dft_result[:N // 2]) # Take the positive
                                                frequencies

        spectrogram.append(magnitude)

spectrogram = np.array(spectrogram).T

# Plot and save the spectrogram
plt.figure(figsize=(12, 8))
plt.imshow(spectrogram, aspect='auto', cmap='magma', origin='lower',
           extent=[0, len(audio) / sr, 0,
                  sr / 2])

plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')

spectrogram_file = Path(output_path) / f"{Path(filename).stem}_spectrogram
                                           .png"

plt.savefig(spectrogram_file)
plt.close()
print(f"Spectrogram saved to {spectrogram_file}")

def analyze_audio(input_path, output_directory):
    # Load audio
    audio, sr = librosa.load(input_path, sr=None)

    # Analyze pitch and harmonics
    pitch_freqs, hnr = analyze_pitch_and_harmonics(audio, sr)

    # Analyze noise
    noise_estimate = analyze_noise(audio)

    # Save analysis results
    save_analysis_results(output_directory, input_path, pitch_freqs, hnr,
                          noise_estimate)

    # Save spectrogram
    save_spectrogram(audio, sr, output_directory, input_path)

    return pitch_freqs, hnr, noise_estimate

def process_directory(input_directory, output_directory):
    if not os.path.exists(output_directory):
        os.makedirs(output_directory)

    for filename in os.listdir(input_directory):
        if filename.endswith(".mp3"):
            input_path = os.path.join(input_directory, filename)

            print(f"Processing {filename}...")

            try:
                pitch_freqs, hnr, noise_estimate = analyze_audio(input_path,
                                                                output_directory)
                print(f"Pitch frequencies (Hz) for {filename}: {pitch_freqs}")
                print(f"Harmonic-to-Noise Ratio for {filename}: {hnr}")
                print(f"Noise Estimate for {filename}: {noise_estimate}")
            except Exception as e:
                print(f"Error processing {filename}: {str(e)}")

    print("Processing complete for all files.")

if __name__ == "__main__":
    # Specifying the directories

```



```

input_directory = r"E:\Python\DSP OUTPUT"
output_directory = r"E:\Python\DSP OUTPUT"

# Process all audio files in the input directory
process_directory(input_directory, output_directory)

```

6.2 Output:

```

Pitch frequencies (Hz): [
3263.3591 3547.1245 3548.8054 3554.9004 3562.1736 3330.253
3542.7634 3546.998 3549.0332 3541.4268 3338.531 3337.7026
3535.9922 3536.9844 3332.3726 3376.1577 3615.385 3354.6711
3369.2754 3719.413 3356.379 3609.7102 3329.6445 3360.4604
3612.4631 3729.652 3729.4646 3734.725 3733.0728 3732.4407
3718.3098 3716.7861 3718.9048 3695.6462 3686.889 3447.9932
3193.8806 3527.757 3608.8438 3640.6223 3640.6267 3638.7122
3646.8955 3702.0574 3723.3252 3765.5508 3657.527 3657.155
3660.4014 3639.0361 3464.5195 3522.0642 3669.2937 3679.1768
3700.1482 3710.9558 3718.7415 3718.2651 3710.0164 3592.5713
3599.5981 3654.109 3645.7302 3647.2437 3615.6946 3628.7422
3640.5376 3638.676 3708.803 3628.8496 3631.7068 3644.8037
3649.6553 3557.9424 3565.165 3728.8755 3727.2927 3665.2224
3672.7156 3657.3093 3616.6143 3599.0688 3608.6003 3638.197
3635.3503 3722.271 3737.681 3532.3179 3534.6484 3661.7505
3660.6099 3649.8396 2656.9158 387.23236 387.52023 386.234
375.60358 360.94714 348.46994 388.32303 396.0054 393.07352
397.5735 406.86075 409.1129 403.80173 396.58063 292.25037
306.34625 314.85944 319.40103 316.01813 438.82443 434.45032
442.07397 375.25076 1600.1132 1599.0815 527.1545 539.814
266.92822 391.67 506.48358 506.98996 514.00085 517.2897
519.25977 518.104 513.57996 537.81476 544.3896 550.9675
560.75726 434.07953 579.6775 580.05566 576.74646 574.56244
704.8214 694.0837 685.02185 551.44904 713.93756 609.1026
610.98254 745.2761 733.84576 723.25616 720.2315 719.37427
1581.6307 1578.2166 1577.7892 1582.4707 724.2433 605.1363
1647.4254 1643.3027 1586.888 3563.62 738.0593 547.90015
546.9243 739.3768 1868.819 1870.1715 604.26825 603.8733
546.83923 545.18304 537.2962 486.95172 495.92496 499.25812
498.68375 2395.047 2395.0676 2553.549 2546.3472 3201.8206
3545.1035 3575.8945 3577.7456 3557.4226 3717.803 3717.9275
3590.0007 3722.6296 3456.8918 3577.1875 3595.7227 3600.46
3600.8762 3726.5789 3610.3955 3598.3694 3690.86 3610.0464
3335.6658 3625.1184 3624.4739 3664.8538 3691.5137 2803.1904
387.49213 373.0468 420.55844 367.63586 355.38345 377.47906
527.0076 503.53665 644.2257 754.3539 753.12866 623.9224
639.5562 606.4986 605.75024 486.62396 605.9341 607.03564
624.72656 611.6473 612.4661 615.57184 610.52356 606.2095
609.8435 609.02625 603.0371 596.72345 601.80695 643.6352
783.76544 1063.1881 1067.1149 796.2388 819.95575 838.0945
839.6953 832.9067 814.62006 547.90814 1013.4218 811.96075
496.26083 505.74728 504.52057 497.31113 494.21112 979.9537
3207.5251 1134.7158 905.9314 902.8788 865.15607 727.15717
734.939 735.8642 732.7417 732.1476 734.34235 488.9207
497.3909 493.0156 477.85583 462.35638 480.86072 459.80896
462.51892 466.6236 469.32416 468.04904 464.6723 460.12347
461.6767 458.62747 455.78308 455.0479 465.2954 468.05133
460.1139 455.49042 452.51266 445.99487 925.51984 1206.5022
873.3761 441.79523 429.05905 470.48105 1061.4401 1068.8463
438.39667 472.62094 937.0834 935.68256 2932.0408 3199.4246
3204.4111 3628.3516 3559.43 3663.4475 3667.0352 3514.2048
3620.145 3612.009 3579.5208 3576.487 3479.9983 3480.1982
3647.776 3652.7898 3665.841 3664.6108 3674.7336 3706.561
3609.4714 3612.7234 3427.3845 3589.1812 3578.403 3583.8557
3669.6252 3588.382 3617.032 3327.9697 3199.7996 3199.5747
3415.2644 3203.5583 3202.813 3565.9585 3622.2734 3622.3306
3619.6926 3335.3323 3469.9219 3443.0027 3593.0986 3645.748
3631.8257 3622.772 3624.361 ]

```

```
Harmonic-to-Noise Ratio: [ 5.0702109e-11  4.1787830e-11 -3.7207428e-11 ... -1.5837078e-07  
-1.3606238e-07 -9.9762055e-08]  
Noise Estimate (std dev): 0.04135514050722122
```

